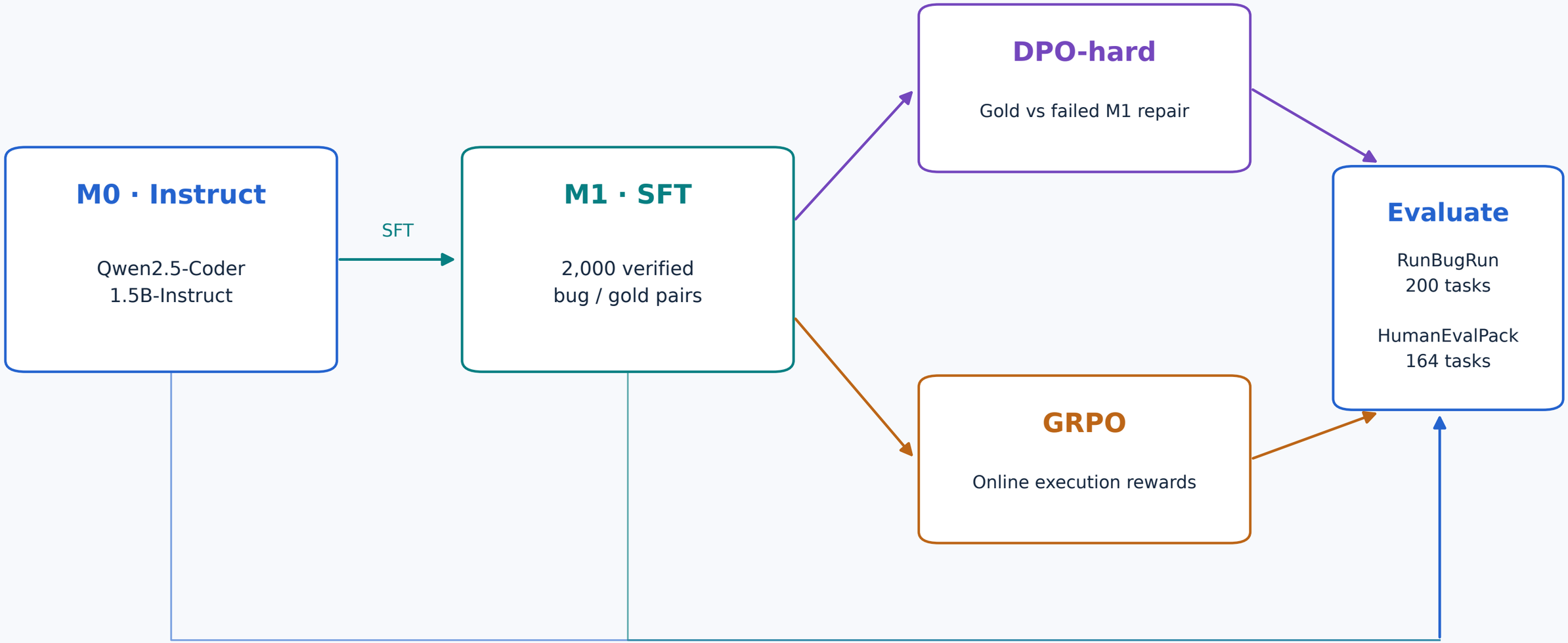
RepairRL: one warm start, two post-training branches

DPO and GRPO independently start from M1. Every model is evaluated with the same V1 protocol.



Additional comparator: untouched pretrained Coder. Agent feedback is inference-only; it does not update model weights.

From published examples to verified training pairs

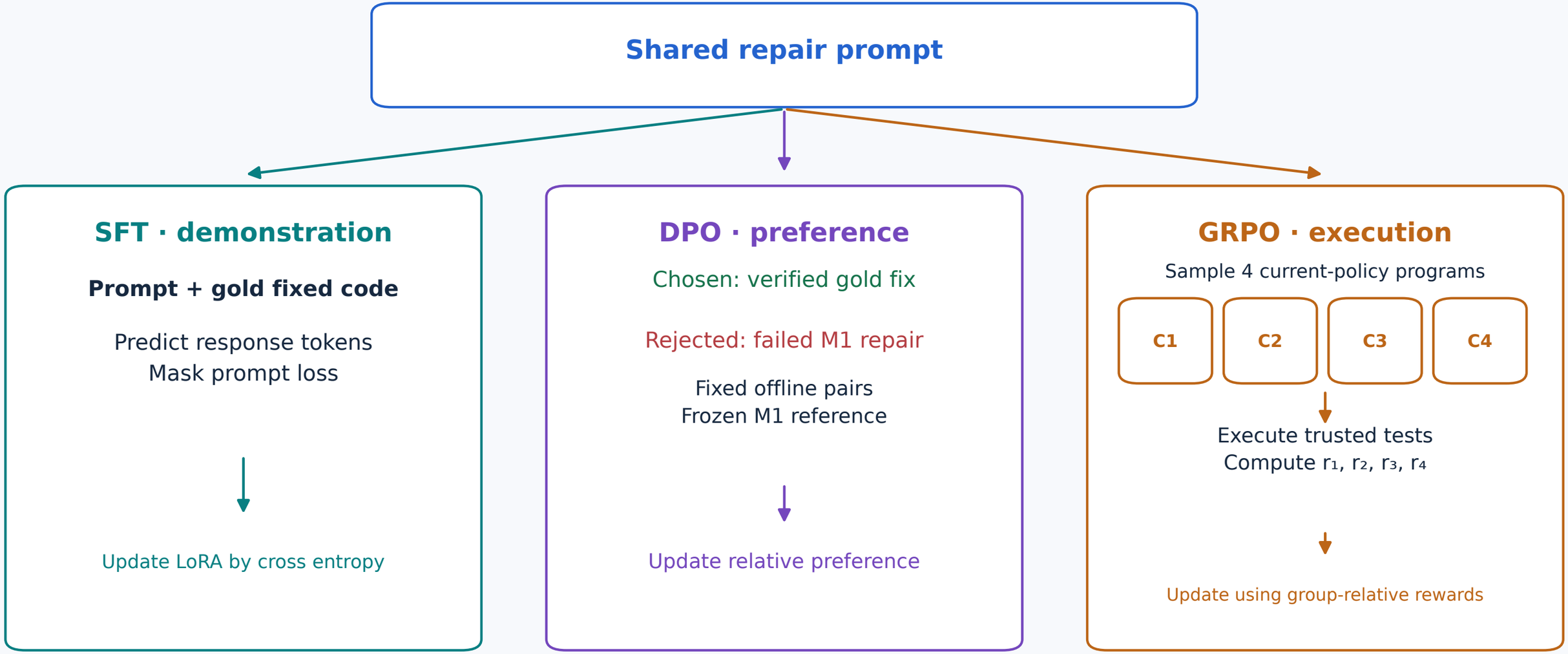
RunBugRun v2 · Python · short self-contained programs · deterministic problem-group splits



Gold passes all selected tests; buggy code fails ≥1. Test suites: 2–10 cases. Train/dev/test problem IDs are disjoint.

Three learning signals for the same repair task

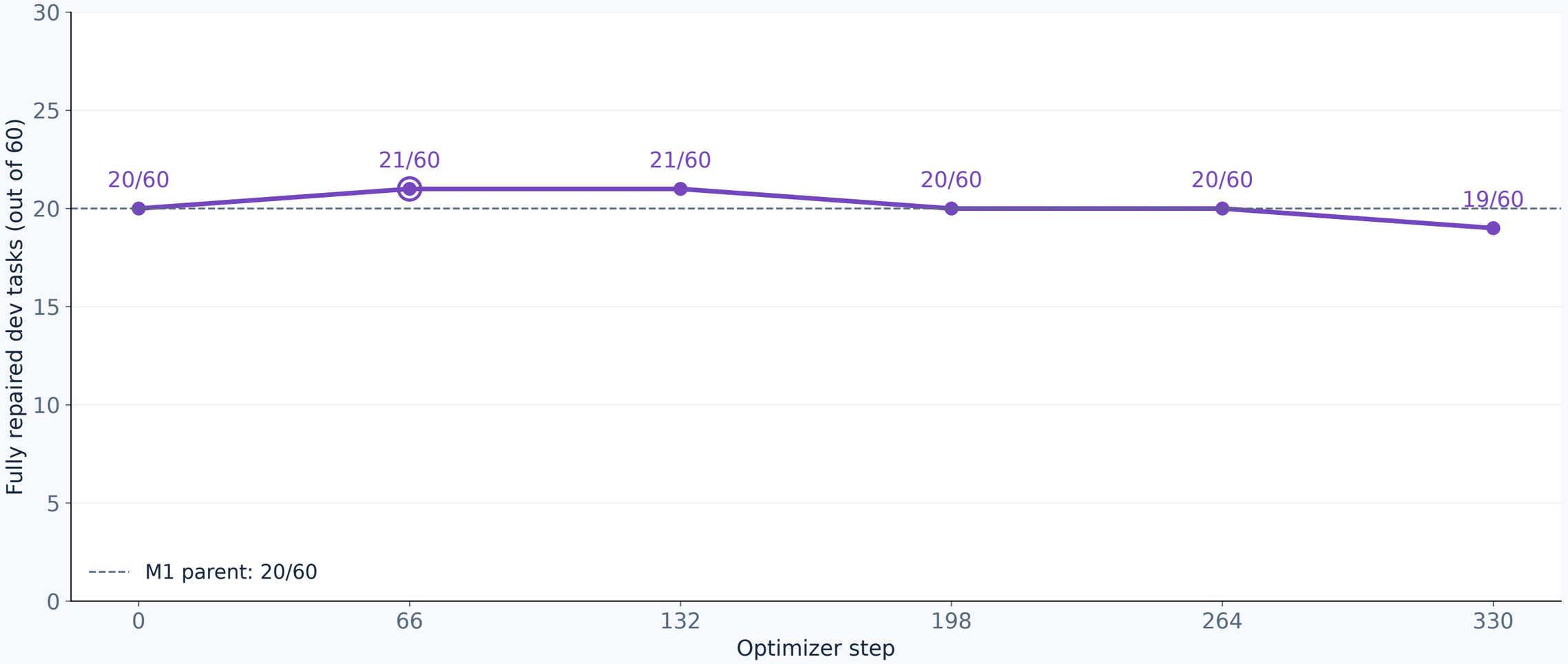
Same input contract: problem specification + buggy Python program → full replacement program



C1–C4 are schematic; rewards come from execution. Hidden tests stay outside the generated program.

DPO dev progression: early peak, later decline

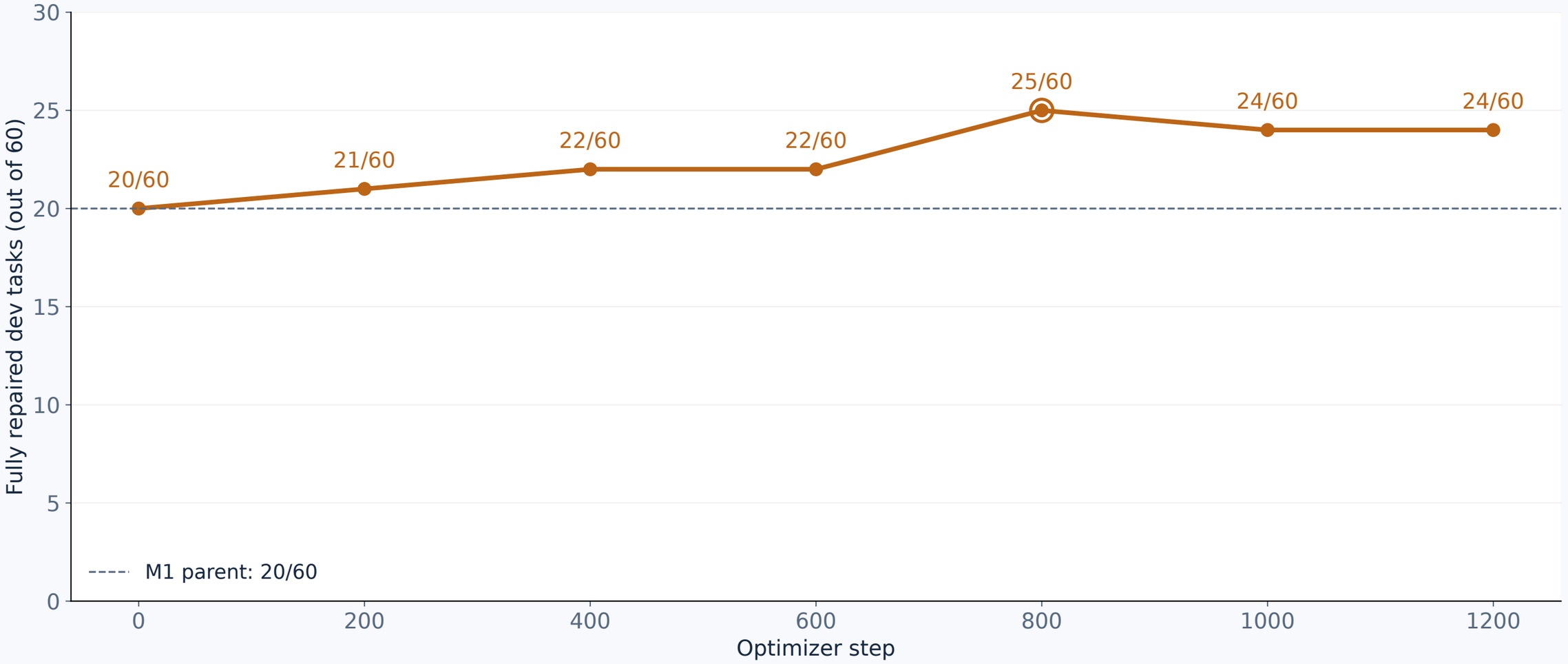
Repair pass@1 on a fixed 60-problem dev set — not training preference accuracy



Selected checkpoint: step 66 (tied at 132; earlier wins). Later dev decline is consistent with overfitting or mismatch, not proof of cause.

GRPO: dev success peaks at step 800

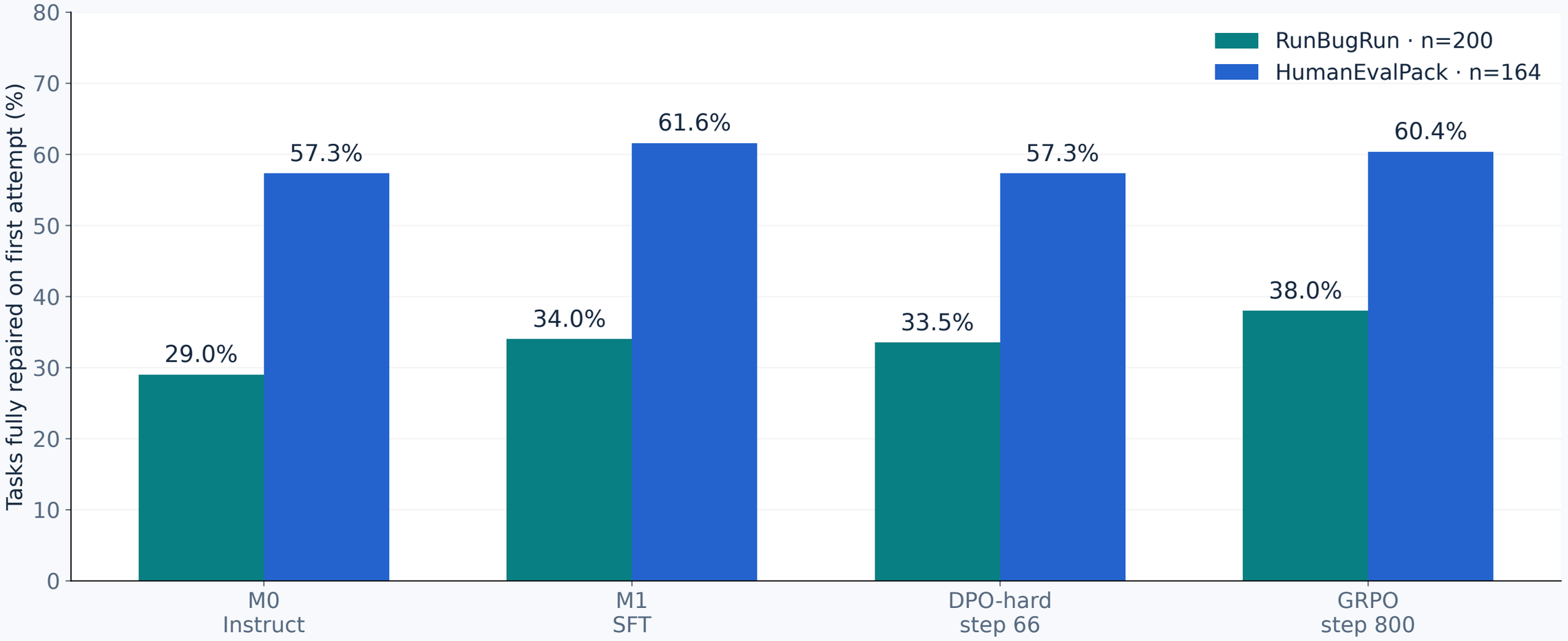
Fixed 60-problem dev screen · snapshots every 200 steps · completed evaluations through step 1,200



Step 800: 25/60 (41.7%). Steps 1,000 and 1,200: 24/60 (40.0%). These are dev results, not the held-out comparison.

Held-out repair: compare the models, not training loss

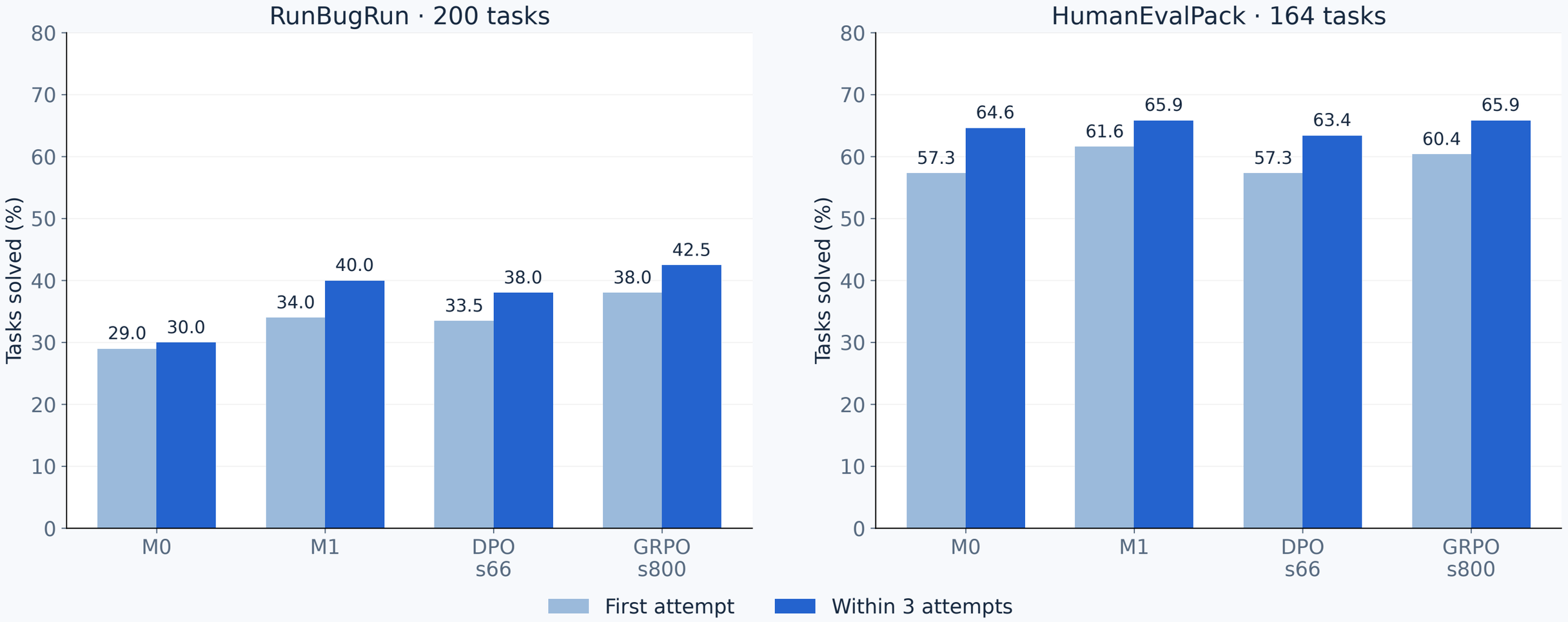
Pass@1 · completed V1 / FP16 evaluations · DPO step 66 and interim GRPO step 800



Benchmarks differ in format and distribution. Single trained configurations; this is not a universal algorithm ranking.

Does execution feedback add successful repairs?

First attempt versus success within at most 3 sequential feedback-guided attempts



Agent@3 is not independent-sample pass@3. V1 retries receive only a sanitized test summary; no hidden inputs or expected outputs.

Actual repair: a cup swap must update the state only once

RunBugRun 21346 / p00047 · measured first attempts · curated example, not a representative sample

Original bug

```
if x == base:
    base = y
if y == base:
    base = x
```

Two independent branches can undo the same swap.

Task: track the ball’s location as cup positions are swapped.

Pretrained — 2/3

```
Two == branches retained
```

M0 Instruct — 0/3

```
range(int(input()))
```

M1 SFT — 1/3

```
Two != branches
```

DPO-hard · step 66 — 3/3

```
if … elif …
```

GRPO · step 800 — 3/3

```
if … elif …
```

Gold / passing repair

```
if x == base:
    base = y
elif y == base:
    base = x
```

DPO & GRPO
3/3 selected tests passed

Code excerpts summarize the recorded full programs. M0 invents a leading count; M1 changes comparisons. Hidden scoring cases are omitted.